

# **LAPORAN PRATIUM PEMROGRAMAN ALGORITMA DAN PEMROGRAMAN**

**“Stack”**



**disusun Oleh:**

**Nama : Alhamd Ridho Putra**

**NIM : 2511532020**

**Dosen Pengampu : Wahyudi, S.T, M.T.**

**Asisten Praktikum : Sherly Sukmadira**

**DEPARTEMEN INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS ANDALAS  
TAHUN 2026**

## KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** dengan materi **Stack (Tumpukan)** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi sekaligus pemahaman atas praktikum yang telah dilaksanakan. Melalui materi *Stack*, penulis belajar mengenai konsep penyimpanan data dengan prinsip **LIFO (Last In, First Out)** serta implementasinya dalam penyelesaian masalah pemrograman.

Penulis juga mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Bapak Wahyudi, S.T, M.T., selaku dosen mata kuliah Struktur Data.
2. Kakak-kakak Asisten Praktikum yang telah membimbing selama jalannya praktikum di laboratorium.
3. Rekan-rekan mahasiswa yang telah bertukar pikiran dalam menyelesaikan tugas ini.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami konsep *Stack*.

Padang, 19 April 2026

Penulis

## DAFTAR ISI

|                                      |                                     |
|--------------------------------------|-------------------------------------|
| <b>KATA PENGANTAR.....</b>           | <b>1</b>                            |
| <b>DAFTAR ISI.....</b>               | <b>2</b>                            |
| <b>BAB I PENDAHULUAN.....</b>        | <b>3</b>                            |
| <b>1.1. Latar Belakang.....</b>      | <b>3</b>                            |
| <b>1.2. Tujuan Praktikum.....</b>    | <b>4</b>                            |
| <b>1.3. Manfaat Praktikum .....</b>  | <b>4</b>                            |
| <b>BAB II PEMBAHASAN.....</b>        | <b>5</b>                            |
| <b>2.1. Uraian Masalah .....</b>     | <b>Error! Bookmark not defined.</b> |
| <b>2.2. Langkah Kerja .....</b>      | <b>Error! Bookmark not defined.</b> |
| <b>2.3. Analisis Hasil .....</b>     | <b>Error! Bookmark not defined.</b> |
| <b>BAB III KESIMPULAN.....</b>       | <b>13</b>                           |
| <b>3.1. Kesimpulan .....</b>         | <b>13</b>                           |
| <b>3.2. Saran Pengembangan .....</b> | <b>13</b>                           |
| <b>DAFTAR PUSTAKA.....</b>           | <b>13</b>                           |

## BAB I

### PENDAHULUAN

#### 1.1. Latar Belakang

Dalam dunia ilmu komputer, struktur data merupakan cara penyimpanan, pengorganisasian, dan pengaturan data di dalam media penyimpanan komputer sehingga data tersebut dapat digunakan secara efisien [1]. Pemilihan struktur data yang tepat sangat krusial dalam pengembangan perangkat lunak karena memengaruhi performa algoritma dan penggunaan memori.

Salah satu struktur data linear yang bersifat fundamental adalah *Stack* atau tumpukan. *Stack* didefinisikan sebagai sekumpulan data yang terorganisir di mana penambahan dan penghapusan elemen hanya dapat dilakukan pada satu ujung yang sama, yang disebut sebagai *top* [2]. Struktur ini beroperasi berdasarkan prinsip *Last In, First Out* (LIFO), di mana elemen yang terakhir dimasukkan akan menjadi elemen yang pertama kali dikeluarkan.

Implementasi *Stack* sangat luas dalam dunia pemrograman, mulai dari manajemen memori (seperti *call stack* pada pemanggilan fungsi), fitur *undo/redo* pada pengolah kata, hingga proses evaluasi ekspresi aritmatika dan algoritma *backtracking* [3]. Melalui praktikum ini, pemahaman mendalam mengenai mekanisme internal *Stack*, termasuk operasi dasarnya seperti *push* dan *pop*, sangat diperlukan agar mahasiswa mampu menyelesaikan kompleksitas permasalahan komputasi dengan lebih terstruktur.

## 1.2. Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum materi *Stack* ini adalah:

1. Memahami konsep dasar dan karakteristik struktur data *Stack* dengan prinsip LIFO.
2. Mampu mengimplementasikan operasi-operasi dasar pada *Stack* seperti *push*, *pop*, *peek*, *isEmpty*, dan *isFull*.
3. Menganalisis perbedaan penggunaan *Stack* baik menggunakan representasi *array* maupun *linked list*.
4. Mampu menerapkan struktur data *Stack* untuk memecahkan masalah logika pemrograman sederhana.

## 1.3. Manfaat Praktikum

Manfaat yang diharapkan setelah mengikuti praktikum ini adalah:

1. Mahasiswa memiliki kemampuan teknis dalam membangun struktur data *Stack* secara mandiri menggunakan bahasa pemrograman.
2. Meningkatkan kemampuan analisis dan logika mahasiswa dalam memilih struktur data yang efisien untuk kasus tertentu.
3. Memberikan pemahaman mendasar sebagai persiapan untuk mempelajari materi struktur data yang lebih kompleks, seperti pohon (*tree*) dan graf (*graph*).

## BAB II

### PEMBAHASAN

#### 2.1. Uraian Masalah

Praktikum ini melibatkan lima implementasi utama terkait struktur data *Stack* menggunakan bahasa pemrograman Java:

1. **latihanStack\_2511532020**: Program ini mendemonstrasikan penggunaan *class* bawaan `java.util.Stack`. Fokus utama adalah pemahaman operasi dasar: `push()` untuk menambah data, `pop()` untuk mengambil sekaligus menghapus elemen teratas, dan `peek()` untuk melihat elemen teratas tanpa menghapusnya.

```
package pekan3_2511532020;

import java.util.Stack;

public class latihanStack_2511532020 {

    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(42);
        s.push(-3);
        s.push(17);
        System.out.println("nilai stack= " +s);
        System.out.println("nilai pop= " +s.pop());
        System.out.println("nilai stack setelah pop= " +s);
        System.out.println("nilai peek= " +s.peek());
        System.out.println("nilai stack setelah peek= " +s);

    }

}
```

2. **NilaiMaksimum\_2511532020**: Permasalahan yang diselesaikan adalah mencari nilai tertinggi di dalam *Stack*. Karena operasi *Stack* bersifat destruktif saat pengambilan data, program ini menggunakan *Stack* bantuan (backup) untuk menyimpan sementara data yang di-*pop*, sehingga urutan data asli tetap terjaga setelah proses pencarian selesai.

```
package pekan3_2511532020;

import java.util.Stack;

public class NilaiMaksimum_2511532020 {
    public static int max(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();
        int maxValue_2511532020 = s.pop();
        backup.push(maxValue_2511532020);
        while (!s.isEmpty()) {
            int next_2511532020 = s.pop();
            backup.push(next_2511532020);
            maxValue_2511532020 = Math.max(maxValue_2511532020,
next_2511532020);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue_2511532020;
    }
    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack: " + s);
        System.out.println("Stack teratas: " + s.peek());
        System.out.println("Nilai Maksimum: " + max(s));
    }
}
```

3. **SiswaStack\_2511532020**: Implementasi ini menunjukkan bahwa *Stack* dapat menyimpan objek kompleks (bukan hanya tipe data primitif). Program menggunakan *ArrayList* sebagai dasar penyimpanan (*underlying storage*) untuk membungkus objek Siswa (Nama dan NIM) ke dalam perilaku *Stack*.

```
package pekan3_2511532020;

import java.util.ArrayList;
class Siswa {
    String nama;
    int nim;
```

```

    public Siswa(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim: " + nim + ", Nama: " + nama;
    }
}

public class SiswaStack_2511532020 {
    private ArrayList<Siswa> stack;

    public SiswaStack_2511532020() {
        stack = new ArrayList<>();
    }

    public void push_2511532020(Siswa mhs) {
        stack.add(mhs);
    }

    public Siswa pop_2511532020() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswa peek_2511532020() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa_2511532020() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }

    public static void main(String[] args) {
        SiswaStack_2511532020 studentStack_2511532020 = new
        SiswaStack_2511532020();

        Siswa mhs1 = new Siswa("Ali", 1);
        Siswa mhs2 = new Siswa("Boby", 2);
        Siswa mhs3 = new Siswa("Charles", 3);

        studentStack_2511532020.push_2511532020(mhs1);
        studentStack_2511532020.push_2511532020(mhs2);
        studentStack_2511532020.push_2511532020(mhs3);

        System.out.println("Siswa di dalam stack: ");
    }
}

```



```

        studentStack_2511532020.tampilkanSiswa_2511532020();

        System.out.println("Siswa teratas " +
studentStack_2511532020.peek_2511532020());
        System.out.println("mengeluarkan siswa teratas dari stack:
" + studentStack_2511532020.pop_2511532020());
        System.out.println("daftar siswa setelah di pop :");
        studentStack_2511532020.tampilkanSiswa_2511532020();
    }
}

```

4. **stackArray\_2511532020**: Merupakan implementasi *Stack* secara manual (*low-level*) menggunakan *array* statis. Di sini, variabel *top* berperan penting sebagai penunjuk indeks terakhir. Program ini juga menangani kondisi *Overflow* (tumpukan penuh) dan *Underflow* (tumpukan kosong).

```

package pekan3_2511532020;

public class stackArray_2511532020 {
    static int MAX = 1000;
    int top;
    int a[] = new int[MAX];
    boolean isEmpty()
    {
        return (top < 0);
    }
    stackArray_2511532020()
    {
        top = -1;
    }
    boolean push_2511532020(int x)
    {
        if (top >= (MAX - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a[++top] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop_2511532020()
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
        }
    }
    int peek_2511532020()

```

```

{
    if (top < 0) {
        System.out.println("Stack Underflow");
        return 0;
    }
    else {
        int x = a[top];
        return x;
    }
}
void print_2511532020() {
    for (int i = top; i > -1; i--) {
        System.out.print(" " + a[i]);
    }
}
}

```

```
package pekan3_2511532020;
```

```

public class stackArrayDriver_2511532020 {
    public static void main(String[] args) {
        stackArray_2511532020 s = new
stackArray_2511532020();
        s.push_2511532020(10);
        s.push_2511532020(20);
        s.push_2511532020(30);
        System.out.println(s.pop_2511532020() + " dikeluarkan
dari stack");
        System.out.println("Elemen teratas adalah : " +
s.peek_2511532020());
        System.out.print("Elemen pada stack :");
        s.print_2511532020();
    }
}
}

```

5. **StackPostfix\_2511532020:** Penerapan praktis *Stack* dalam mengevaluasi ekspresi aritmatika *postfix* (misal: 5 2 4 \* + 7 -). *Stack* digunakan untuk

menyimpan *operand*, sementara operator akan memicu operasi aritmatika terhadap dua elemen teratas.

```
package pekan3_2511532020;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511532020 {
    public static int postfixEvaluate(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input_2511532020 = new Scanner(expression);
        while(input_2511532020.hasNext()) {
            if (input_2511532020.hasNextInt()) {
                s.push(input_2511532020.nextInt());
            } else {
                String operator_2511532020 =
input_2511532020.next();
                int operand2_2511532020 = s.pop();
                int operand1_2511532020 = s.pop();
                if (operator_2511532020.equals("+")) {
                    s.push(operand1_2511532020 +
operand2_2511532020);
                } else if (operator_2511532020.equals("-"))
{
                    s.push(operand1_2511532020 -
operand2_2511532020);
                } else if (operator_2511532020.equals("*"))
{
                    s.push(operand1_2511532020 *
operand2_2511532020);
                } else {
```

```

s.push(operand1_2511532020 /
operand2_2511532020);
    }
}
}
input_2511532020.close();
return s.pop();
}
public static void main(String[] args) {
    System.out.println("hasil postfix= " + postfixEvaluate("5 2
4 * + 7 -"));
}
}
}

```

## 2.2. Langkah Kerja

Berikut adalah urutan langkah yang dilakukan selama praktikum:

1. Menyiapkan lingkungan pengembangan (*Integrated Development Environment*) seperti Eclipse, IntelliJ, atau VS Code.
2. Membuat *package* dengan nama pekan3\_2511532020.
3. Mengimplementasikan *class* Siswa sebagai cetak biru data objek.
4. Menuliskan kode program untuk masing-masing kelas sesuai dengan skenario (penggunaan *library* vs implementasi manual).
5. Melakukan kompilasi program untuk memastikan tidak ada kesalahan sintaks (*syntax error*).
6. Menjalankan program (*running*) dan mengamati *output* pada konsol untuk setiap skenario pengujian.
7. Melakukan proses *debugging* jika hasil *output* tidak sesuai dengan prinsip LIFO.

### 2.3. Analisis Hasil

Berdasarkan hasil eksekusi kode di atas, didapatkan beberapa poin analisis sebagai berikut:

- **Prinsip LIFO dan Integritas Data:** Pada kelas `NilaiMaksimum`, terlihat bahwa untuk memproses seluruh data dalam *Stack*, kita harus mengeluarkan elemen satu per satu. Penggunaan *Stack* tambahan terbukti efektif untuk menjaga integritas data agar *Stack* asli kembali ke kondisi semula setelah pemrosesan selesai [4].
- **Efisiensi Penyimpanan:** Implementasi menggunakan *array* pada `stackArray_2511532020` memiliki kelebihan dalam kecepatan akses karena lokasi memori yang berdekatan (*contiguous*). Namun, terdapat kelemahan pada batasan ukuran tetap (*fixed size*). Sebaliknya, penggunaan `ArrayList` pada `SiswaStack` memberikan fleksibilitas ukuran secara dinamis [5].
- **Evaluasi Ekspresi:** Pada `StackPostfix`, *Stack* berhasil menyederhanakan logika perhitungan tanpa memerlukan tanda kurung. Hal ini menunjukkan bahwa *Stack* adalah struktur data yang paling efisien untuk memproses hierarki operasi dalam kompilator [6].
- **Catatan Temuan (Bug Syntax):** Pada kode `stackArray_2511532020`, terdapat logika `if (top >= (MAX = 1))`. Secara teknis, ini akan menyebabkan *Stack Overflow* terlalu dini karena `MAX` diatur ulang menjadi 1. Idealnya, pengecekan dilakukan dengan `if (top >= MAX - 1)` untuk memanfaatkan kapasitas penuh *array*.
- **Kompleksitas:** Secara teoritis, akses elemen menggunakan `get()` pada `ArrayList` memiliki kompleksitas waktu  $O(1)$ , namun operasi penambahan di tengah atau penghapusan memiliki kompleksitas  $O(n)$  karena adanya pergeseran elemen [2].

## **BAB III**

### **KESIMPULAN**

#### **3.1. Kesimpulan**

Berdasarkan hasil praktikum dan analisis yang telah dilakukan terhadap berbagai implementasi *Stack*, dapat ditarik beberapa kesimpulan sebagai berikut:

1. *Stack* adalah struktur data linear yang bekerja dengan prinsip **LIFO (*Last In, First Out*)**, di mana data yang terakhir masuk akan menjadi yang pertama kali diproses atau dikeluarkan.
2. Operasi dasar *Stack* meliputi push (menambah data), pop (menghapus data teratas), dan peek (melihat data teratas). Penggunaan metode pop bersifat destruktif terhadap urutan data, sehingga diperlukan mekanisme penyimpanan sementara (*backup stack*) jika ingin memproses data tanpa kehilangan struktur aslinya.
3. Implementasi *Stack* di Java dapat dilakukan melalui *library* standar (`java.util.Stack`), pemanfaatan `ArrayList`, maupun pembuatan secara manual menggunakan *array* statis. Masing-masing memiliki karakteristik performa dan fleksibilitas memori yang berbeda.
4. *Stack* terbukti sangat efektif dalam menangani logika yang membutuhkan urutan balik atau hierarki, seperti evaluasi ekspresi *postfix* dan pengelolaan objek data kompleks.

#### **3.2. Saran Pengembangan**

Untuk pengembangan lebih lanjut dalam pemahaman materi *Stack*, penulis menyarankan beberapa hal berikut:

1. **Implementasi Linked List:** Praktikum selanjutnya sebaiknya mencoba mengimplementasikan *Stack* menggunakan *Linked List* untuk mengatasi keterbatasan ukuran tetap (*fixed size*) yang ada pada *array* statis [7].

2. **Visualisasi GUI:** Pengembangan program dapat ditingkatkan dengan membuat antarmuka grafis (GUI) yang memvisualisasikan proses tumpukan secara *real-time* agar lebih mudah dipahami oleh pengguna awam.
3. **Optimasi Algoritma:** Disarankan untuk mengeksplorasi penggunaan *Stack* dalam algoritma yang lebih kompleks, seperti konversi ekspresi *infix* ke *postfix* atau algoritma pencarian jalur (*pathfinding*) [5].

## TUGAS PRAKTIKUM

### 1. KODE ADT

```
// Nama kelas wajib menggunakan NIM lengkap [cite: 8, 13]
public class Website_2511532020 {
    // Variabel atribut menggunakan 4 digit terakhir NIM [cite: 10, 11]
    private String judul_2020;
    private String url_2020;

    // Constructor untuk inisialisasi [cite: 18]
    public Website_2511532020(String judul, String url) {
        this.judul_2020 = judul;
        this.url_2020 = url;
    }

    // Selektor (Getter) [cite: 19]
    public String getJudul_2020() {
        return judul_2020;
    }

    public String getUrl_2020() {
        return url_2020;
    }

    // Mutator (Setter) [cite: 20]
    public void setJudul_2020(String judul) {
        this.judul_2020 = judul;
    }

    public void setUrl_2020(String url) {
        this.url_2020 = url;
    }
}
```

### 2. KELAS

```
import java.util.Stack;
import java.util.Scanner;
import java.util.InputMismatchException;
```



```

public class Browser_2511532020 {
    public static void main(String[] args) {
        // Menggunakan Stack dengan tipe data Website [cite: 39]
        Stack<Website_2511532020> history_2020 = new Stack<>();
        Scanner input = new Scanner(System.in);
        int pilihan = 0;

        do {
            try {
                System.out.println("\n= Browser History NIM:
2511532020");

                System.out.println("1. Kunjungi Website (Push)");
                System.out.println("2. Tombol Back (Pop)");
                System.out.println("3. Lihat Halaman Aktif (Peek)");
                System.out.println("4. Keluar");
                System.out.print("Pilihan: ");

                pilihan = input.nextInt();
                input.nextLine(); // Membersihkan buffer

                switch (pilihan) {
                    case 1:
                        // Operasi Push [cite: 22]
                        System.out.print("Masukkan Judul: ");
                        String judul = input.nextLine();
                        System.out.print("Masukkan URL: ");
                        String url = input.nextLine();

                        history_2020.push(new
Website_2511532020(judul, url));
                        System.out.println("Berhasil mengunjungi
halaman!");

                        break;

                    case 2:
                        // Operasi Pop dengan penanganan kondisi
Stack kosong [cite: 23, 42]

```

```

        if (!history_2020.isEmpty()) {
            Website_2511532020 dihapus =
history_2020.pop();
            System.out.println("Berhasil menghapus:
" + dihapus.getJudul_2020());
        } else {
            System.out.println("History kosong
(Gagal Pop)!");
        }
        break;

    case 3:
        // Operasi Peek dan Cek Status
        if (!history_2020.isEmpty()) {
            Website_2511532020 aktif =
history_2020.peek();
            System.out.println("Halaman Saat Ini: "
+ aktif.getJudul_2020());
            System.out.println("URL: " +
aktif.getUrl_2020());
            System.out.println("Total Riwayat: " +
history_2020.size());
        } else {
            System.out.println("Riwayat masih
kosong!");
        }
        break;

    case 4:
        System.out.println("Program selesai.");
        break;

    default:
        System.out.println("Pilihan tidak valid!");
    }
} catch (InputMismatchException e) {
    // Mencegah error jika user memasukkan huruf saat
menu pilihan

```

```
        System.out.println("Error: Masukkan angka saja!");  
        input.nextLine(); // Membersihkan input yang salah  
        pilihan = 0;  
    }  
} while (pilihan != 4);  
  
input.close();  
}  
}
```

## DAFTAR PUSTAKA

- [1] A. Tenenbaum, Y. Langsam, and M. Augenstein, *Data Structures Using C*. New Delhi: Prentice-Hall of India, 1990.
- [2] R. Lafore, *Data Structures and Algorithms in Java*, 2nd ed. Indianapolis: Sams Publishing, 2002.
- [3] M. T. Goodrich, R. Tamassia, and D. Mount, *Data Structures and Algorithms in Python*. Hoboken, NJ: John Wiley & Sons, 2013.
- [4] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston: Pearson, 2012.
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [6] G. F. Isaacs, "Stack-based Evaluation of Arithmetic Expressions," *Journal of Computing Sciences in Colleges*, vol. 17, no. 5, pp. 10-21, 2002.
- [7] S. Sahni, *Data Structures, Algorithms, and Applications in Java*. Silicon Press, 2004.